

Motus2: A Self-Evolving General World Model for Dexterous Manipulation

Hongzhe Bi^{1,2*†}, Zihao Zhou^{1,3*}, Yihang Tang^{1,4*}, Jingrui Pang^{1,2*}, Shuhe Huang^{1,2*}, Haitian Liu^{1,2},
Runqing Wang^{1,4}, Shuai Huang¹, Yichen Wang¹, Yiming Cheng², Ruowen Zhao^{1,2}, Zhenghua Li²,
Hengkai Tan^{1,2}, Xiaolong Liu¹, Jinhui Wan¹, Jiabao Liu¹, Min Zhao^{1,2}, Fan Bao¹, Jun Zhu^{1,2}✉

¹GensPI ²Tsinghua University ³BUAA ⁴BIT

[†]Project lead. *Joint first authors. ✉Corresponding author.

bhz24@mails.tsinghua.edu.cn; dcszj@tsinghua.edu.cn

Abstract

General embodied agents should perceive, predict, act, evaluate, and improve within a unified system. World models have shown great promise in building such agents, yet existing models typically append an action output head to a world simulator, without coupling them into a closed decision-and-learning loop for policy improvement. We present **Motus2**, a self-evolving general world model for dexterous manipulation. Motus2 advances world modeling through model scaling and data scaling. For **model scaling**, a single model with shared weights exposes three control interfaces: a policy (*world-action model*), a simulator (*action-conditioned world model*), and an evaluator (*value model*). The policy proposes candidate action chunks, the simulator predicts their visual consequences, and the evaluator assesses the predicted outcomes. Their coupling forms a closed decision-and-learning loop for policy improvement. This formulation uses curated expert demonstrations for action learning, while failed and suboptimal interactions provide valuable evidence for dynamics modeling and value learning. For **data scaling**, Motus2 progresses from large-scale monocular egocentric data to synchronized stereo egocentric data, followed by robot-domain adaptation with robot trajectories and supplementary human-robot alignment data. Motus2 further studies global-autoregressive and hybrid-memory extensions of its sliding-window context, adds tactile feedback for contact-aware control, and is instantiated on a fully biomimetic platform with stereo vision, dual arms, dual dexterous hands, and tactile sensing. Together, egocentric data scaling and closed-loop general world model scaling provide a general path toward self-evolving dexterous manipulation.

Project Page: <https://motus-robotics.github.io/motus2>.

1. Introduction

General-purpose dexterous manipulation requires more than a policy that maps the latest observation to an action. A capable model should learn scalable interaction priors, retain task-relevant history under self-occlusion, reason about contact, and improve its policy beyond supervised imitation. Yet today’s robot foundation models are trained predominantly on curated action-supervised datasets [5, 6, 8, 24, 25, 26, 28]. Collecting embodiment-aligned robot demonstrations at scale is expensive, while imitation alone provides no notion of whether an action is good or bad and no mechanism for improving the policy from its own outcomes. This motivates a self-evolving formulation in which the model predicts and evaluates the consequences of candidate actions and uses the resulting feedback for policy improvement.

Egocentric human data provides a scalable source of dexterous interaction priors. Monocular egocentric data provide broad coverage of diverse tasks and hand-object interactions [20, 21, 29], while synchronized stereo egocentric data additionally provide implicit depth cues and support more accurate 3D hand-pose estimation. However, transferring these priors to dexterous robot execution remains challenging. Contact-

critical events such as fingertip slip, grasp formation, and release are often ambiguous from vision alone, while first-person manipulation is partially observable because the hands occlude objects and relevant consequences may appear only later. These challenges motivate robot-domain adaptation, temporal context, and tactile feedback [22, 30, 31].

On the model side, recent control-oriented world models increasingly provide two key capabilities. A *world-action model* proposes executable actions and can therefore serve as a policy, while an *action-conditioned world model* predicts their consequences and can therefore serve as a simulator. Motus unifies these capabilities through UniDiffuser-style joint video-action modeling within a single shared model [1, 4]. This reflects a key principle of General World Models: action is not an auxiliary output attached to a simulator, but the causal interface that grounds internal predictions in physical interaction [9]. However, predicted consequences alone do not indicate whether an outcome advances the task. Self-evolution therefore requires a third interface: an evaluator, realized by a value model, that assesses predicted outcomes and provides signals for policy improvement. The policy, simulator, and evaluator determine what action to attempt, what will happen under that action, and whether the predicted outcome is desirable, respectively. Therefore, the question is *How to realize these causal interfaces within one shared model without exposing future observations to action prediction, and to convert outcome values into policy updates?*

To answer the above question, we present Motus2, a **self-evolving General World Model for dexterous manipulation** built on Motus. Motus2 scales dexterous manipulation along two axes: *data scaling* through a hierarchical egocentric human-data pyramid and *model scaling* through a general world model with an explicit policy improvement closed-loop. Specifically, for data scaling, pretraining progresses from large-scale monocular egocentric data to synchronized stereo egocentric data, followed by robot-domain mid-training on robot trajectories supplemented with human-robot alignment data. For model scaling, one shared-parameter model exposes a policy implemented by a world-action model, a simulator implemented by an action-conditioned world model, and an evaluator implemented by a value model. The policy proposes candidate action chunks, the simulator predicts their visual consequences, and the evaluator estimates their task progress. Best-of- N planning uses these estimates for test-time selection, while model-based reinforcement learning with DiffusionNFT [3] converts them into policy updates, closing the decision-and-learning loop required for self-evolution.

This formulation separates imitation targets from other forms of interaction evidence. Curated demonstrations supervise action learning, whereas failed and suboptimal interactions provide evidence for dynamics modeling and value learning. A lightweight tactile expert further supports tactile-conditioned action refinement and tactile prediction for contact-sensitive execution. To study long-horizon partial observability, we compare two extensions of the bounded sliding-window context: global autoregression and hybrid memory [30].

In summary, our contributions are:

- **A General World Model for dexterous manipulation.** Motus2 jointly models executable action chunks, action-conditioned future observations, and task-progress values within a shared-parameter video-action model. The model exposes three control interfaces: a policy implemented by a world-action model, a simulator implemented by an action-conditioned world model, and an evaluator implemented by a value model. A lightweight tactile expert further supports tactile-conditioned action refinement and tactile prediction for contact-sensitive execution (§3.1, §3.2, §3.5).
- **An egocentric data-scaling and robot-domain transfer study.** Pretraining progresses from monocular egocentric data to stereo egocentric data, followed by robot-domain mid-training on robot trajectories and supplementary human-robot alignment data. Scaling experiments further establish a stereo human-data scaling trend (§3.2, §4, §5.3).
- **A value-guided closed-loop self-evolution method.** The policy proposes candidate actions, the simulator predicts their consequences, and the evaluator estimates their task progress. Model-based reinforcement learning converts the resulting scores into policy updates, while the same interfaces support Best-of- N test-time planning. Failed and suboptimal interactions provide evidence for dynamics modeling and value learning (§3.3).

2. Related Work

General World Models The term *world model* covers systems with substantially different representations and control capabilities, including latent dynamics for control, generative prediction of observable futures, and interactive generation conditioned on external inputs [32, 33, 34]. *General World Model* (GWM) provide an overall framework for building a general foundation: it integrates an understanding of the current world, imagination of possible and action-conditioned futures, and action grounded by feedback from the external world [9]. We therefore distinguish the complete GWM from the individual functional interfaces through which a model predicts, acts, or evaluates outcomes.

Within robot control, we organize recent systems by three functional interfaces. The *policy* interface is realized by a *world-action model* (WAM), which generates actions from observations and task context. Representative models such as Motus, DreamZero, Fast-WAM, MotuBrain, Being-H0.7, and Dyna-2 couple video modeling with action generation for control [1, 7, 18, 35, 36, 37]. The *simulator* interface is realized by an *action-conditioned world model*, which predicts future observations under a proposed action; representative examples include DreamDojo, Ctrl-World, and GigaWorld [19, 38, 39]. The *evaluator* interface is realized by a *value model* [40], reward model, or preference model that assesses predicted outcomes.

In particular, Motus realizes the policy and simulator interfaces as conditional query modes of one UniDiffuser-style video-action model, supporting video-action joint prediction, VLA, inverse dynamics, video generation, and action-conditioned future prediction with shared parameters [1, 4]. Motus2 develops this foundation for dexterous manipulation by adding a value-based evaluator and model-based reinforcement learning, thereby coupling policy, simulation, and evaluation within one shared-parameter GWM. At a broader methodological level, control-oriented world models build on advances in efficient diffusion sampling and scalable generative architectures [10, 11, 12, 13], as well as diffusion-based behavior modeling and value-guided policy optimization [3, 14, 15, 16, 17].

Dexterous Policies from Egocentric and Human Priors Dexterous manipulation requires high-DoF hands to coordinate discontinuous contact dynamics, in which small pose or timing errors can alter the grasp mode, induce slip, or prevent recovery. Recent systems address these challenges through hand-specific action representations, retargeting, tactile-reactive control, and scalable data generation, as in Dexora, DexMimicGen, T-Rex, and METIS [22, 23, 27, 41]. Thus, dexterity depends not only on policy capacity but also on embodiment-aware supervision and contact feedback.

Egocentric human data provides scalable interaction priors across monocular and stereo observation settings. Monocular egocentric data provides broad coverage of diverse environments, objects, and human interaction behaviors, while stereo egocentric data additionally provides implicit depth cues from synchronized binocular observations and more accurate 3D hand-pose estimates. H-RDT, EgoVLA, and EgoScale demonstrate how human interaction data can strengthen robot policy learning, while EgoSteer further leverages human-robot alignment for robot execution [2, 20, 21, 42]. Being-H0.7 and EgoWAM add world-aware objectives over egocentric data [36, 43].

World-Model-Based Policy Improvement Model-based RL learns a dynamics model to predict future outcomes, thereby providing supervision for policy improvement. Recent VLA post-training methods extend beyond action imitation: WMPO optimizes policies with world-model objectives, RISE uses a compositional world model for self-improvement, Reinforcing Action Policies by Prophesying learns from future-conditioned feedback, and NORA-1.5 constructs world-model and action-based preference rewards [44, 45, 46, 47]. These methods show that predicted consequences can provide supervision beyond expert demonstrations, especially when real robot interaction is limited. An open question is how action decision-making, world simulation, and value estimation can be coupled within a unified model to exploit the complementary supervision provided by heterogeneous interaction data—including expert demonstrations, successful trajectories, suboptimal executions, and failures—for policy, dynamics, and value learning.

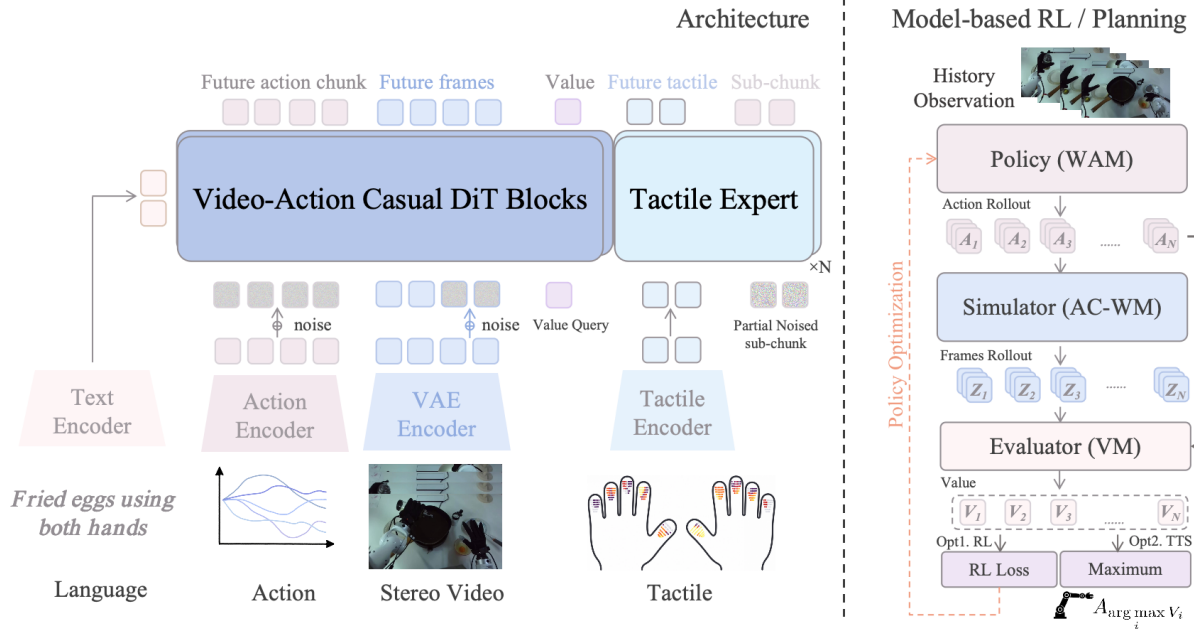


Figure 1: Motus2 overview. A single general world model with one shared parameter set exposes three control interfaces. The policy generates action chunks from language, current robot observations, and the selected visual working-memory context; the simulator predicts future visual states under candidate actions; and the evaluator assesses the resulting branches for planning and policy optimization.

3. Method

3.1. Overview

Fig. 1 summarizes Motus2. We first learn a shared video–action backbone through joint egocentric pre-training, then use action-first factorization and trajectory-dependent supervision in robot-domain training to expose policy, simulator, and evaluator interfaces from the same parameters. Value-guided MBRL closes these interfaces into a learning loop, while working memory and the tactile expert extend visual context and contact feedback, respectively.

Formally, we consider the general task of language-conditioned dexterous manipulation and formulate it as a partially observable Markov decision process (POMDP):

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{T}, \Omega, r, \gamma), \quad (1)$$

where $s_t \in \mathcal{S}$ is the latent physical state, $o_t = (I_t, q_t, \tau_t) \in \mathcal{O}$ contains vision, proprioception, and optional tactile sensing, and $a_t \in \mathcal{A}$ is the robot action. $\mathcal{T}$, Ω , r , and γ denote transition dynamics, the observation model, reward, and discount. Because the hands frequently occlude objects and contacts, the current observation is not a sufficient Markov state. We therefore condition the deployed policy on the observation history $h_t = (\ell, o_0, \dots, o_t)$ under language instruction ℓ . In practice, the core model represents h_t by a context c_t containing language, current proprioception, and an encoded visual history supplied by one of the working-memory mechanisms in §3.4; the tactile expert uses the concurrent tactile stream as described in §3.5.

Given this context, let $A_t = (a_t, \dots, a_{t+H-1})$ be an executable action chunk, Z_t its latent future observations, and Y_t a discretized task-progress value. The support of Y_t consists of the numerical centers of the progress bins. Starting at robot-domain mid-training, we factor their shared density in the same action-first order used for control, planning, and policy optimization:

$$p_\theta(A_t, Z_t, Y_t | c_t) = \underbrace{\pi_\theta(A_t | c_t)}_{\text{policy (WAM)}} \cdot \underbrace{p_\theta^{\text{wm}}(Z_t | c_t, A_t)}_{\text{simulator (AC-WM)}} \cdot \underbrace{p_\theta^{\text{vm}}(Y_t | c_t, A_t, Z_t)}_{\text{evaluator (VM)}}. \quad (2)$$

These factors are three interfaces of the same shared-parameter model, not separately trained architectures. The branch-ranking score is the evaluator’s conditional expectation

$$V_\theta(c_t, A_t, Z_t) = \mathbb{E}_{Y_t \sim p_\theta^{\text{ym}}(\cdot | c_t, A_t, Z_t)} [Y_t]. \quad (3)$$

This factorization defines the desired control chain; the remaining problem is to realize it from a jointly pretrained video–action backbone without splitting the three factors into separate networks.

3.2. General World Modeling

We address this challenge during robot-domain adaptation while preserving the shared backbone. A stage-specific mask implements the policy–simulator–evaluator dependency in Eq. (2), while trajectory-dependent loss gates route each trajectory only to the factors it can validly supervise.

Stage-specific information flow. Joint pre-training learns the video–action representation with mutual within-chunk visibility and no value query. When robot-domain mid-training begins, we introduce a read-only value query and organize each window as clean teacher-forced observation history followed by action-first chunk blocks

$$\mathbf{x} = (Z^{\text{ctx}}; B_1; \dots; B_M), \quad B_j = (q_j; A_j; Z_j; U_j), \quad (4)$$

where q_j is the optional proprioception, A_j is the action chunk, Z_j is its future video, and U_j is a read-only value query. Action tokens cannot read future-video or value tokens from their own chunk; future-video tokens may read the current action; and the value query may read both but is hidden from all other tokens. Across blocks, the mask is causal and windowed: block j may use preceding clean observations but never a later block. We call this design *chunk-autoregressive*: autoregression occurs across action chunks, while the low-level actions within each chunk are generated jointly by flow matching.

The resulting within-chunk dependency is $A_j \rightarrow Z_j \rightarrow U_j$; tokenization, backbone, and cross-chunk causality remain unchanged (Fig. 2). Both stages use the same backbone initialized from a foundation video diffusion model [48]. Stereo video latents share temporal and vertical coordinates but occupy distinct horizontal RoPE ranges; proprioception, action, and value queries enter the same transformer computation, while language enters through cross-attention.

Trajectory-dependent supervision routing. The mask realizes the required dependency, but cannot determine whether a recorded action should be imitated. Failed and suboptimal executions provide valid transition and outcome evidence, but not desirable action targets. We therefore gate each trajectory to the factors it can validly supervise. For a clean target $x \in \{Z_j, A_j\}$ and Gaussian noise $\epsilon \sim \mathcal{N}(0, I)$, we define

$$x_\sigma = (1 - \sigma)x + \sigma\epsilon, \quad v^* = \epsilon - x, \quad \sigma \in [0, 1]. \quad (5)$$

Our shared core predicts video and action velocity fields v_θ^z and v_θ^a , together with a categorical value distribution p_θ^{ym} . We train them with the unified objective

$$\mathcal{L} = w_z \|v_\theta^z - v_z^*\|_2^2 + \lambda_a w_a \|v_\theta^a - v_a^*\|_2^2 + \lambda_v w_v \text{CE}(p_\theta^{\text{ym}}, Y_t), \quad (6)$$

where the loss gates (w_z, w_a, w_v) and independent noise levels (σ^z, σ^a) select among three training modes under the action-first mask:

mode	(σ^z, σ^a)	(w_z, w_a, w_v)	supervised conditional
policy	$(> 0, > 0)$	$(1, 1, 0)$	$p_\theta(A_t, Z_t c_t)$
simulation	$(> 0, 0)$	$(1, 0, 0)$	$p_\theta(Z_t c_t, A_t)$
evaluation	$(0, 0)$	$(0, 0, 1)$	$p_\theta(Y_t c_t, A_t, Z_t)$

(7)

From mid-training onward, policy mode jointly supervises action and future video under the first two factors of Eq. (2), but deployment reads only its action factor $\pi_\theta(A_t | c_t)$. Simulation mode keeps the recorded action

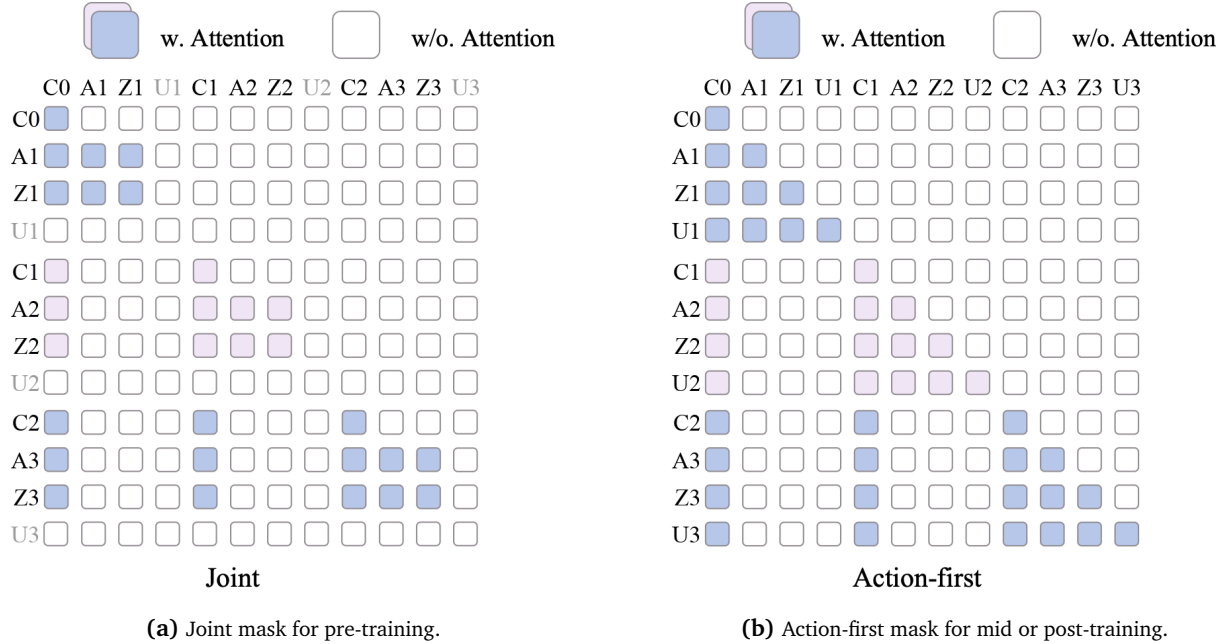


Figure 2: Stage-specific chunk masks. Context (C), video (Z), and action (A) tokens share one backbone; a read-only value query (U) is introduced from mid-training onward and is inactive during joint pre-training. Joint pre-training permits bidirectional video-action interaction within a chunk. The action-first mask blocks current future-video information from action prediction, while future-video tokens may read the action and the value query may read both. Both masks remain causal across chunks.

clean and supervises only action-conditioned future prediction, whereas evaluation mode keeps action and video clean and supervises only the value readout.

From mid-training onward, only curated successful trajectories activate action supervision. We route failed and suboptimal trajectories, together with task-irrelevant interactions, to the applicable simulation or evaluation mode, where their recorded actions remain clean conditioning variables rather than imitation targets. We thereby learn from their transitions and outcomes without teaching the policy to reproduce undesirable behavior.

Data-scaling curriculum. The curriculum first learns joint human-interaction priors and introduces the action-first mask and supervision routing only when robot-domain adaptation begins. During *pre-training*, Stage 1 learns the video pathway from low- and then high-resolution monocular egocentric clips using bidirectional conditional flow matching with one or two clean latent frames as context. Stage 2 introduces synchronized stereo egocentric data and jointly trains the left view, right view, and action tokens with bidirectional within-chunk visibility. During *mid-training*, we ground these human priors in the robot domain using robot trajectories supplemented by human-robot alignment data. At this point, we replace the joint interaction with the action-first mask and mix the three modes in Eq. (7). During *post-training*, we retain this interface for target-robot SFT and study three extensions built on it: value-guided MBRL, long-history context, and tactile feedback, described in §3.3, §3.4, and §3.5, respectively.

Action-first inference. Once trained, the factorization lets ordinary control query only the action factor $\pi_\theta(A_t | c_t)$ and materialize future video and value only when planning or policy optimization requires them. Ordinary control therefore stops after the policy factor, whereas planning and policy improvement activate the full policy-simulator-evaluator chain. This retains joint video-action training without imposing a future rollout on every control step [35, 49].

3.3. Value-Guided Closed-Loop Self-Evolution

The action-first chain makes candidate generation, simulation, and evaluation executable in sequence, but self-evolution further requires the evaluator’s feedback to improve the policy. We train the evaluator with progress-based supervision: successful trajectory segments provide positive progress targets, whereas failed and task-irrelevant interactions provide negative supervision for value learning. The resulting value estimates support both Best-of- N selection and MBRL updates to the action pathway. Because the same shared-parameter model realizes all three interfaces, Motus2 uses its own predicted consequences and value estimates to improve its policy (Fig. 3).

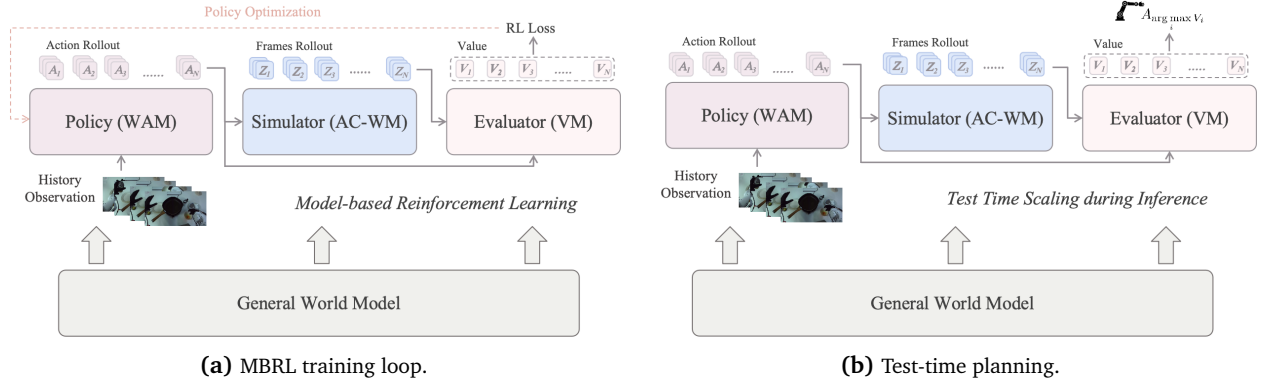


Figure 3: Self-evolution and planning modules. The Motus2 MBRL path samples action candidates with the world–action model (policy), predicts their future visual consequences with the action-conditioned world model (simulator), and evaluates the resulting branches with the value model (evaluator). The resulting values guide DiffusionNFT-style policy updates, while the same three interfaces support test-time planning by selecting the highest-value branch.

3.3.1. Progress-Based Value Learning

To instantiate this progress-based supervision, we follow the relative-progress formulation of VLAC [50] and assign each segment from a successful trajectory the target

$$r_t = \frac{\Delta t}{T - t}, \quad (8)$$

where Δt is the segment length, T is the total trajectory length, and t is its starting timestep. The target lies in $[0, 1]$ and measures the progress of the current segment relative to the remaining task horizon. This provides temporally localized supervision for evaluating action segments at different stages of execution.

The positive targets above teach the evaluator how successful behavior advances, but not how deviations from the task should be scored. We therefore complement them with negative supervision from two sources. Failed teleoperation trajectories capture realistic errors such as inaccurate manipulation, unintended contact, and deviations from the instructed procedure. Task-irrelevant interactions provide clear negative examples of behavior unrelated to the instructed task.

Following the relative-progress formulation, each negative trajectory segment is assigned

$$r_t = -\frac{\Delta t}{T - t}, \quad (9)$$

so negative segments represent behavior that does not contribute to task completion. We discretize the resulting positive and negative rewards into the categorical target Y_t used by the evaluator in Eq. (6).

3.3.2. Planning and Policy Optimization

With the evaluator defined above, the policy–simulator–evaluator chain supports both action selection and policy improvement. At inference, value estimates rank policy candidates for Best-of- N planning; during MBRL, the same estimates guide updates to the policy.

Model-Based Planning. At inference, we reuse the three interfaces as a receding-horizon planner: propose N chunks, simulate and score each branch, execute the highest-valued chunk, append the real observation, and repeat. Replanning after each real observation limits reliance on long imagined rollouts. For branch i , the simulator predicts Z_i and the evaluator returns $V_i := V_\theta(c_t, A_i, Z_i)$. We select $i^* = \arg \max_{i=1,\dots,N} V_i$ and execute A_{i^*} .

Policy Optimization. Planning improves selection but cannot change the proposal distribution. We therefore feed evaluator scores back into the action pathway with DiffusionNFT [3], which converts scalar branch scores into a flow-matching policy update while keeping the simulator and evaluator fixed.

For every generated candidate pair (A_i, Z_i) with value V_i , we sample a noise level σ_i and Gaussian noise ϵ_i and apply the forward flow-matching process

$$A_i^{\sigma_i} = (1 - \sigma_i)A_i + \sigma_i\epsilon_i, \quad v_i^{\text{tar}} = \epsilon_i - A_i, \quad (10)$$

where v_i^{tar} is the flow-matching velocity target associated with the sampled clean action and noise.

The online policy and the EMA reference policy predict action velocity fields $v_{\theta,i}$ and $v_{\text{ref},i}$ from the same noised action, noise level, language instruction, robot state, and clean observation history. The predicted future latent Z_i affects the policy update through its value V_i , but is not exposed to the action-velocity predictor during NFT optimization.

We convert the scores of the policy-generated candidates into optimality weights:

$$\hat{r}_i = 0.5 + 0.5 \cdot \text{clip} \left(\frac{V_i - \text{mean}(V)}{\max(\text{std}(V), \varepsilon)}, -1, 1 \right), \quad (11)$$

where $V = (V_1, \dots, V_N)$, and $\text{mean}(V)$ and $\text{std}(V)$ are the mean and population standard deviation of the generated candidate values, respectively. The constant $\varepsilon > 0$ prevents numerical instability when their variance is zero or near zero. The resulting weight satisfies $\hat{r}_i \in [0, 1]$: candidates valued above the mean receive $\hat{r}_i > 0.5$, whereas candidates valued below the mean receive $\hat{r}_i < 0.5$.

Each candidate group additionally contains one ground-truth action–future pair. It is assigned $\hat{r}_i = 1$ as a positive behavioral anchor and is excluded from the mean and standard-deviation calculation over policy-generated candidates.

Following DiffusionNFT, we construct two implicit velocity fields around the reference prediction:

$$v_i^+ = (1 - \beta)v_{\text{ref},i} + \beta v_{\theta,i}, \quad v_i^- = (1 + \beta)v_{\text{ref},i} - \beta v_{\theta,i}, \quad (12)$$

where the reference prediction is held fixed during each update. The coefficient $\beta > 0$ controls the displacement of the implicit fields around the reference policy, with $1/\beta$ acting as the corresponding reinforcement-guidance strength.

The action policy is optimized using

$$\mathcal{L}_{\text{NFT}} = \mathbb{E}_i \left[\hat{r}_i \|v_i^+ - v_i^{\text{tar}}\|_2^2 + (1 - \hat{r}_i) \|v_i^- - v_i^{\text{tar}}\|_2^2 \right]. \quad (13)$$

High-value candidates emphasize the positive implicit field, moving the online action distribution toward the corresponding actions. Low-value candidates emphasize the negative implicit field, moving the online distribution away from them. Candidates with $\hat{r}_i \approx 0.5$ provide little directional preference and keep the online policy close to the reference.

Thus, planning changes selection from the current policy distribution, whereas MBRL updates the policy distribution from which future candidates are drawn.

Asynchronous MBRL Training. The update above repeatedly executes candidate rollout, value scoring, reference construction, and policy optimization. These stages have mismatched throughput, so serial execution would leave expensive components idle. We therefore implement a Ray-based macro-asynchronous pipeline, inspired by AcceRL [51], that decouples offline prefix sampling, imagined rollout and value scoring, reference-target construction, and policy optimization through bounded FIFO buffers. Rollout actors generate candidate actions, predict their visual consequences, and attach value scores. For each candidate, a reference actor samples the flow-matching noise, constructs $A_i^{\sigma_i}$ and v_i^{tar} , and evaluates the same noised action with a versioned EMA policy to obtain the fixed reference prediction $v_{\text{ref},i}$. The trainer then evaluates $v_{\theta,i}$ with the online policy and applies Eq. (13). Only action-related parameters are updated; the shared video backbone and evaluator remain frozen. We tag samples with the EMA reference version and discard stale samples or samples delayed beyond the configured bound.

3.4. Working Memory

The closed loop above can use only the evidence retained in c_t . A bounded context may discard evidence observed early in a task before it is needed again; hands may also hide the object in later views. We therefore make accessible history an explicit design axis. A bounded sliding window is our fixed-cost streaming default, and we implement two post-trained long-history extensions—global autoregression and hybrid working memory—that preserve earlier evidence in different forms.

Sliding window. We use fixed-length windows throughout joint pre-training and retain them under the later action-first layout. From mid-training onward, each window contains clean teacher-forced observations and the action-first prediction chunks defined in §3.2. A prediction chunk may attend to preceding clean observation chunks within the window, but never to future chunks. An intermediate observation can therefore serve both as the prediction target of its current chunk and, in clean form, as context for later chunks. We vary the visible history width during training so that the policy does not rely on a single fixed context length.

At deployment, we retain observed rather than imagined visual latents so the context remains anchored to the executed trajectory. The KV cache keeps only the most recent clean observations. After an action chunk is executed, the new observation is appended and the oldest one is evicted when the cache is full. Temporal RoPE is window-relative: after eviction, the retained video keys are rebased so that their temporal coordinates remain within the positional range used in training. Both cache capacity and per-step attention cost are therefore bounded with respect to episode length, although observations that leave the window are no longer accessible. The window settings are reported in Appendix E.

Global autoregression. To remove the information loss caused by eviction, our global-autoregressive variant retains all preceding clean visual latents. We use episode-level temporal coordinates rather than rebasing positions within a window, allowing each prediction chunk to attend to the full observation history. Its KV cache and attention cost consequently grow with episode length.

Hybrid working memory. To preserve selected long-range evidence without retaining every old frame at full resolution, we integrate a MemoryWAM-style hybrid memory [30]. It retains initial anchor frames and a recent observation window at full resolution, while representing older intermediate observations with persistent memory tokens.

Variable-length episode packing. The two long-history variants must learn across complete trajectories rather than truncated windows, producing highly variable sequence lengths that make fixed-count batching inefficient. We therefore train them on complete episodes and pack those episodes by latent-frame load.

Specifically, we pack multiple episodes into one attention call using a block-diagonal causal mask. No query can attend across episode boundaries; rotary coordinates are generated independently for each episode, and padding tokens contribute no loss. Packing therefore changes only the computational layout and is

equivalent to processing the episodes independently. The offline packing plan balances total latent-frame load across micro-batches while keeping every episode intact; an episode that exceeds the nominal frame budget is processed as an untruncated singleton. The packing limits and optimization settings are provided in Appendix E.

These mechanisms trade fixed-cost streaming, full-history retention, and compressed persistence; we compare the two long-history extensions in §5.5.

3.5. Tactile Expert: Tactile Refinement and Prediction

Working memory extends the available visual history, but vision may still leave contact state ambiguous. To incorporate high-rate tactile evidence without rerunning the full backbone, we introduce a lightweight tactile expert that reuses the backbone’s intermediate action chunk and detached layer-wise key-value (KV) cache. Following T-Rex [22], the backbone denoises the full action chunk from $\sigma = 1$ to a fixed σ_c once; immediately before each short sub-chunk is executed, the expert uses the latest tactile window to complete its $\sigma_c \rightarrow 0$ update.

Tactile-conditioned action refinement. At decision time t , let $A_{t,k}^{\sigma_c}$ denote intermediate sub-chunk k , $\tau_{t,k}^{\text{pre}}$ the tactile window immediately preceding its execution, and $KV_t = \{KV_t^l\}_{l=1}^L$ the cached visible-context and refreshed action K/V. The tactile expert, parameterized by ϕ , reads $A_{t,k}^{\sigma_c}$, $\tau_{t,k}^{\text{pre}}$, and the corresponding detached cache to predict the final action velocity $v_{\phi,t,k}^a$. The refined sub-chunk is

$$\tilde{A}_{t,k} = A_{t,k}^{\sigma_c} - \sigma_c v_{\phi,t,k}^a. \quad (14)$$

The same intermediate full action chunk and KV_t are reused across the rolling updates; after one sub-chunk is executed, the newly acquired tactile window conditions the next.

With $A_{t,k}$ denoting the clean action sub-chunk, the refinement loss is

$$\mathcal{L}_{\text{ref}} = \mathbb{E}_{t,k} \left[\left\| v_{\phi,t,k}^a - \frac{A_{t,k}^{\sigma_c} - A_{t,k}}{\sigma_c} \right\|_2^2 \right]. \quad (15)$$

The intermediate action and KV_t are detached when optimizing the expert.

Tactile prediction. Action refinement uses the preceding tactile window to correct the next sub-chunk. To also supervise how contact evolves under that action, we predict the real force window $\mathbf{f}_{t,k}^{\text{post}}$ following each action sub-chunk. Keeping the preceding tactile window clean, we sample $\epsilon_{t,k}^f \sim \mathcal{N}(0, I)$ and $\sigma_{t,k}^f \in [0, 1]$ and apply the same flow-matching objective:

$$\begin{aligned} \mathbf{f}_{t,k}^{\sigma^f} &= (1 - \sigma_{t,k}^f) \mathbf{f}_{t,k}^{\text{post}} + \sigma_{t,k}^f \epsilon_{t,k}^f, \\ \mathcal{L}_{\text{pred}} &= \mathbb{E}_{t,k,\sigma^f,\epsilon^f} \left[\left\| v_{\phi,t,k}^f - \left(\epsilon_{t,k}^f - \mathbf{f}_{t,k}^{\text{post}} \right) \right\|_2^2 \right], \end{aligned} \quad (16)$$

Here $v_{\phi,t,k}^f$ is decoded in normalized force-signal space. Future-force tokens may attend to intermediate action tokens, whereas action-refinement tokens cannot attend to the noised future-force tokens, preserving the action-first dependency. For the Sharpa Wave, deformation images are clean conditioning inputs only and are not predicted. The complete objective is $\mathcal{L}_{\text{tac}} = \mathcal{L}_{\text{ref}} + \lambda_{\text{pred}} \mathcal{L}_{\text{pred}}$. Force prediction is used only during training; deployment outputs the refined actions. Architecture and training settings are provided in Appendix E.

4. Large-Scale Egocentric Dataset

We construct an approximately 130K-hour egocentric corpus from open-source and procured data spanning monocular and stereo cameras, multiple resolutions, diverse manipulation tasks, and varied environments.

Table 1 reports approximate source-level raw hours before quality filtering, temporal segmentation, and offline annotation. The monocular portion provides broad visual and semantic coverage, while the stereo portion additionally provides implicit depth cues and more accurate 3D hand-pose estimates.

Table 1: Composition of the egocentric corpus. Report durations are approximate raw recording hours before filtering, segmentation, and annotation.

Dataset	Camera Setup	Raw Hours	Resolution ($W \times H$)	Acquisition
Egocentric-100K	Monocular	100,500	480×360	Open-source
Egocentric-10K	Monocular	10,000	$480 \times 360, 512 \times 512$	Open-source
EgoVerse	Monocular	1,200	512×512	Open-source
EgoDex	Monocular	800	$480 \times 360, 512 \times 512$	Open-source
Ropedia	Stereo	7,000	512×512	Mixed
EgoScale	Stereo	6,000	640×384	Procured
LightWheel	Stereo	1,200	640×480	Procured
JD-Group	Stereo	2,000	512×512	Procured
CyberOrigin	Stereo	1,200	640×416	Procured
Total	Mixed	130,000	Mixed	Mixed

Separate from the corpus above, we collect tens of hours of human–robot alignment data using Wuji Human Gloves as supplementary mid-training data. Mid-training uses more than 100 hours of data in total. Collection details are provided in Appendix B.

Pretraining follows the two-stage organization in §3.2. Stage 1 uses low-resolution monocular data for 500K steps and then high-resolution monocular data for 340K steps, while Stage 2 introduces multi-resolution stereo egocentric data for 450K steps. Compatible sources may contribute to multiple phases; for example, monocular LightWheel recordings enter Stage 1, whereas its stereo recordings enter Stage 2.

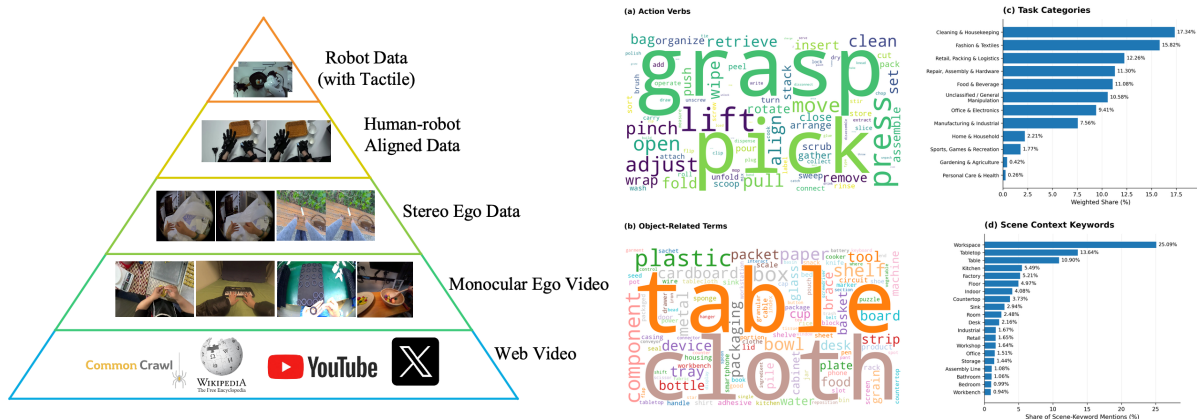


Figure 4: Egocentric data pyramid and corpus semantics. Left: the data hierarchy from web and monocular egocentric video to stereo egocentric data, human–robot alignment data, and robot interaction data. Right: semantic diversity of the annotated subset, excluding Egocentric-100K: (a) 99 fine-grained action verbs, (b) object-related terms, (c) 12 mutually exclusive high-level task categories, and (d) the top 20 non-exclusive scene-context keywords.

The annotations span manipulation primitives, objects, tasks, and scenes. Frequent actions include grasp, pick, lift, press, and adjust; task categories cover cleaning, textiles, logistics, assembly, food preparation, office work, manufacturing, household activities, recreation, gardening, and personal care. All recordings

are converted into synchronized episodes using a common 134-D hand-pose representation, quality-aware temporal segmentation, and vision-language annotation. The complete processing protocol is provided in Appendix C.

5. Experiments

In this section, we aim to answer the following research questions through our experiments:

RQ1: *How does egocentric human pretraining affect downstream robot control?*

RQ2: *Do MBRL and test-time planning improve performance beyond supervised imitation?*

RQ3: *How do different context mechanisms affect long-horizon control?*

RQ4: *Does the tactile expert improve contact-rich manipulation?*

5.1. Experiment Setup

Task suites. The main suite contains five target-robot tasks spanning diverse control requirements: Place Ball and Put Phone test spatial grounding and placement; Attach Eraser and Screw Bulb require precise object alignment and sustained contact; and Multi-Finger tests coordinated multi-finger grasping and manipulation of differently shaped objects. The MBRL study uses Put Phone and Multi-Finger to isolate the effects of policy optimization and test-time planning on the same physical tasks. The memory study uses the separate Find Square and Press Button probes, whose relevant evidence may be occluded or delayed beyond a short observation window. This separation prevents the broad manipulation suite from obscuring the specific effect of persistent context. The tactile study uses Pull Out the Paper Cup and Tear Paper, which require contact-sensitive grasp adjustment and sustained bimanual interaction.

Evaluation protocol. Within each evaluation setting, all methods use the same initial configurations for each task. Real-robot evaluations use 20 configurations per task, whereas the simulation branch of the long-horizon context study uses 25. Each method executes one closed-loop rollout per configuration, using identical language instructions, episode horizons, and task-specific success checkers. A rollout is counted as successful only when all required task predicates are satisfied within the horizon; partial completion is counted as failure. We report success rate (SR), with the number of rollouts specified for each study. The average is a macro-average over tasks, so each task receives equal weight. Simulation and real-robot results are reported separately.

Controlled comparisons. We compare Motus2 against $\pi_{0.5}$ [26] under matched target-task SFT data, observation interface, and evaluation protocol. WAN-SFT, Pretrain-SFT, and Motus2 (Midtrain-SFT) use the same SFT procedure and differ only in initialization: WAN, the two-stage egocentric-pretraining checkpoint, and the robot-domain mid-trained checkpoint, respectively. Within each controlled study, all other factors are held fixed.

5.2. Main Results

Table 2: Main results. Each entry reports success rate over 20 rollouts under matched target-robot finetuning. WAN-SFT, Pretrain-SFT, and Midtrain-SFT differ only in initialization.

Method	Place Ball	Multi-Finger	Attach Eraser	Screw Bulb	Put Phone	Avg. SR $\uparrow$
$\pi_{0.5}$	0%	0%	0%	0%	0%	0%
WAN-SFT	0%	0%	0%	0%	0%	0%
Pretrain-SFT	60%	35%	90%	55%	15%	51%
Motus2 (Midtrain-SFT)	100%	70%	100%	90%	60%	84%

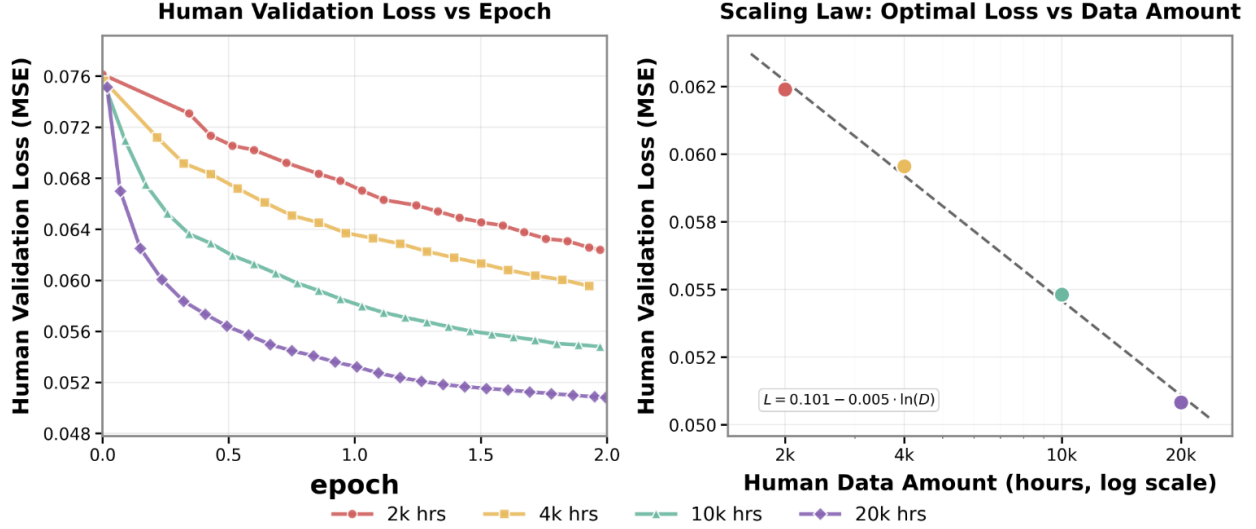


Figure 5: Scaling laws of stereo egocentric human data. Left: held-out human-action prediction error across training epochs for models trained on subsets constructed from 2k, 4k, 10k, and 20k raw recording hours of stereo data. Right: optimal validation error versus raw stereo human-data hours on a logarithmic scale, together with the fitted log-linear scaling law. The reported hours are measured before quality filtering and temporal segmentation; the effective amount of processed training data is therefore smaller.

Results. Egocentric pretraining raises the macro-average success rate from 0% for WAN-SFT to 51% for Pretrain-SFT. Motus2 (Midtrain-SFT) further increases the average to 84%, a 33-point gain over Pretrain-SFT. Motus2 reaches 100% on Place Ball and Attach Eraser, 90% on Screw Bulb, 70% on Multi-Finger, and 60% on Put Phone. Under the matched target-task SFT protocol, these comparisons support the contributions of egocentric pretraining and subsequent robot-domain mid-training.

5.3. Scaling Laws of Stereo Egocentric Human Data

Prior work has reported action-prediction scaling trends with monocular egocentric human data [21, 37]. We study the corresponding trend for synchronized stereo observations and dexterous human actions using nested 2k, 4k, 10k, and 20k-hour subsets. The subsets preserve the source and resolution proportions, and all checkpoints are evaluated on the same trajectory-disjoint stereo held-out set.

Held-out human-action prediction error serves as the measure of generalization. For each validation sample, the flow-matching policy generates multiple action chunks. Their mean prediction is compared with the normalized ground-truth wrist and hand actions using mean squared error. Let $\mathcal{L}_{\text{val}}(D, e)$ denote the validation error after epoch e for a model pretrained on D hours of data. The optimal validation error at each data scale is

$$\mathcal{L}_{\text{val}}^*(D) := \min_e \mathcal{L}_{\text{val}}(D, e). \quad (17)$$

Fitting the optimal error against data hours yields the log-linear relation

$$\mathcal{L}_{\text{val}}^*(D) \approx 0.101 - 0.005 \ln D. \quad (18)$$

Figure 5 presents the training dynamics and optimal validation errors. As the pretraining corpus grows from 2k to 20k hours, the validation curves shift consistently downward: smaller datasets plateau earlier, whereas larger datasets attain lower errors at comparable epochs. Across the four data scales, the optimal validation error decreases monotonically and is approximately linear in the logarithm of corpus size, indicating that the scaling trend previously observed for monocular egocentric data also extends to stereo observations within the measured range.

5.4. Model-Based Policy Improvement and Test-Time Planning

Factorial design. Starting from the same target-robot checkpoint, we independently enable DiffusionNFT policy optimization and Best-of- N planning. This yields a 2×2 comparison of Motus2, Motus2 + Planning, Motus2 + MBRL, and Motus2 + MBRL + Planning.

Table 3: MBRL and planning results. Each entry reports success rate over 20 rollouts. All variants start from the same target-robot checkpoint. MBRL updates the policy weights, whereas planning changes only inference.

Method	Put Phone	Multi-Finger	Avg. SR $\uparrow$
Motus2	60%	70%	65.0%
Motus2 + Planning	65%	70%	67.5%
Motus2 + MBRL	65%	80%	72.5%
Motus2 + MBRL + Planning	70%	80%	75.0%

Results. Planning raises Motus2 from 65.0% to 67.5% average success, showing a 2.5-point gain from online candidate selection without changing the policy weights. Motus2 + MBRL reaches 72.5% under direct inference, a 7.5-point gain over Motus2. Combining MBRL and planning yields 75.0%. Planning therefore remains beneficial after MBRL (2.5 points), while MBRL remains beneficial with planning enabled (7.5 points), showing observed gains from policy optimization and test-time selection in the evaluated tasks.

Qualitative value trajectories. Appendix F visualizes predicted task progress along one successful and three failed Screw Bulb trajectories. The successful trajectory rises from 0.34 to approximately 0.81, whereas the failed trajectories decrease after task progress is lost. These examples illustrate the signal used for candidate ranking.

5.5. Long-Horizon Context Mechanisms

Protocol. On the Find Square and Press Button probes, we compare global autoregression and hybrid working memory as described in §3.4. The two variants use matched training and paired evaluation configurations in simulation and on the real robot.

Table 4: Evaluation of long-horizon context mechanisms. Simulation success rates are computed over 25 rollouts and real-robot rates over 20. Tasks are the long-horizon memory probes Find Square (find_the_square) and Press Button (press_button).

Setting	Context mechanism	Find Square	Press Button	Avg. SR $\uparrow$
Simulation	Hybrid memory	64%	40%	52%
	Global autoregression	84%	72%	78%
Real robot	Hybrid memory	30%	20%	25.0%
	Global autoregression	65%	50%	57.5%

Results. Global autoregression outperforms hybrid memory on both probes in simulation, reaching 84% on Find Square and 72% on Press Button, compared with 64% and 40% for hybrid memory. Its simulation macro-average is therefore 78%, versus 52% for hybrid memory. The same ordering holds on the real robot: global autoregression achieves 65% and 50% on the two tasks, whereas hybrid memory reaches 30% and 20%, giving macro-averages of 57.5% and 25%, respectively. Global autoregression performs better than hybrid memory in both simulation and real-robot evaluations.

5.6. Effect of Tactile Feedback

Protocol. Both variants start from the same mid-trained checkpoint and use the same target-task post-training data. The w/o Tactile variant removes the tactile expert, whereas the w/ Tactile variant uses the complete refinement and prediction objective in §3.5. We evaluate both variants on Pull Out the Paper Cup with the Sharpa Wave and Tear Paper with Wuji Hand 2.

Table 5: Ablation of tactile feedback from the same mid-trained checkpoint. Each entry reports the success rate over 20 rollouts.

Variant	Pull Out Paper Cup	Tear Paper	Avg. SR $\uparrow$
w/o Tactile	65%	55%	60.0%
w/ Tactile	75%	70%	72.5%

Results. The tactile expert improves success on Pull Out the Paper Cup from 65% to 75% and on Tear Paper from 55% to 70%. The macro-average increases from 60.0% to 72.5%, a 12.5-point gain. These results indicate that tactile feedback is beneficial for fine-grained, contact-sensitive manipulation in the evaluated tasks.

6. Conclusion

We presented Motus2, a self-evolving General World Model for dexterous manipulation. Motus2 treats action generation, action-conditioned future simulation, and outcome evaluation as different conditional functions of a shared physical-world model rather than as separate systems. With one shared set of weights, the model operates as a world-action model that proposes executable actions, an action-conditioned world model that predicts their visual consequences, and a value model that evaluates the predicted outcomes. Coupling these functions forms a closed decision-and-learning loop, enabling model-based policy improvement through candidate generation, consequence prediction, value-based selection, and policy updates.

On the data side, Motus2 treats egocentric human interaction as a scalable source of physical experience and progresses from large-scale monocular observation to stereo egocentric data and robot-domain grounding. The framework uses different interaction trajectories according to the supervision they provide: curated successful demonstrations supervise action learning, whereas failed and suboptimal interactions provide valuable evidence for learning dynamics and outcome evaluation. Tactile feedback further strengthens physical grounding during contact-sensitive manipulation. Together, data scaling through egocentric human interaction and model scaling through closed-loop world modeling provide a concrete path toward self-evolving General World Models for dexterous manipulation.

7. Limitations and Future Work

Limitations. The primary limitation of Motus2 is the scalability of wearable tactile supervision across embodiments. Whether worn by a human or a robot, a tactile glove deforms as the hand changes configuration. Even in the absence of external contact, material strain and internal fabric contact produce noisy tactile signals that are difficult to distinguish from meaningful physical contact. More fundamentally, current dexterous hands are not yet geometrically isomorphic to human hands: their little fingers are often elongated, and some are substantially thicker or larger than human hands. Consequently, a single glove pattern cannot fit both human and robotic hands, so tactile data collected with a human glove cannot be directly transferred to a dexterous hand wearing an embodiment-specific glove. This morphology-induced gap limits the scale and reliability of cross-embodiment tactile learning.

Future Work. From the perspective of *data scaling*, advances in wearable tactile materials and fabrication could enable scalable, whole-hand tactile sensing for fully multimodal egocentric human data collection.

Continued progress in anthropomorphic dexterous hand design could further narrow the morphological gap between robot and human hands, bringing robot interaction data closer to human data and enabling near-isomorphic cross-embodiment learning. From the perspective of *model scaling*, stronger video foundation models and unified multimodal models can provide richer generative priors, longer-horizon predictive dynamics, and more faithful multimodal simulation. Building General World Models on these foundations could unify visual, language, action, tactile, and value signals within a common generative and decision-making process, predict the consequences of candidate actions over extended horizons, and continually improve from large-scale interaction.

Acknowledgments

Hardware Acknowledgments. The robotic hands used in this work were commercially procured from *WUJI* and *SHARPA*, while the robotic arms and robotic platforms were commercially procured from *Tianji*. We are especially grateful to *Yunzhe Pan*, *Jun Chen*, *Zhen Wang* and *Bo Liu* at *WUJI* for their substantial technical support throughout the development of this project.

Data Acknowledgments. We acknowledge *Ropedia*, *EgoScale*, *LightWheel*, *JD-Group*, and *CyberOrigin* as the providers of the commercially acquired datasets used in this work.

Core Contributors

Hongzhe Bi: Base Model, Pre-Training, MBRL, Memory, Tactile Sensing, Sharpa & WuJi-2 Hardware, Real Robot Post-Training, Ego Data, Writing.

Zihao Zhou: MBRL, Memory, Simulation, Real Robot Post-Training, Sharpa & WuJi-2 Hardware, Writing.

Yihang Tang: Sharpa & WuJi-2 Hardware, Tactile Sensing, Real Robot Post-Training.

Jingrui Pang: WuJi-1 Hardware, Human Data Collection, Real Robot Post-Training.

Shuhe Huang: MBRL, Simulation, Writing.

References

- [1] Hongzhe Bi, Hengkai Tan, Shenghao Xie, Zeyuan Wang, Shuhe Huang, Haitian Liu, Ruowen Zhao, Yao Feng, Chendong Xiang, Yinze Rong, Hongyan Zhao, Hanyu Liu, Zhizhong Su, Lei Ma, Hang Su, and Jun Zhu. Motus: A unified latent action world model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 35101–35113, June 2026.
- [2] Hongzhe Bi, Lingxuan Wu, Tianwei Lin, Hengkai Tan, Zhizhong Su, Hang Su, and Jun Zhu. H-rdt: Human manipulation enhanced bimanual robotic manipulation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.
- [3] Kaiwen Zheng, Huayu Chen, Haotian Ye, Haoxiang Wang, Qinsheng Zhang, Kai Jiang, Hang Su, Stefano Ermon, Jun Zhu, and Ming-Yu Liu. DiffusionNFT: Online diffusion reinforcement with forward process. In *International Conference on Learning Representations*, 2026. Oral.
- [4] Fan Bao, Shen Nie, Kaiwen Xue, Chongxuan Li, Shi Pu, Yaole Wang, Gang Yue, Yue Cao, Hang Su, and Jun Zhu. One transformer fits all distributions in multi-modal diffusion at scale. In *International Conference on Machine Learning*, pages 1692–1717, 2023.
- [5] Songming Liu, Lingxuan Wu, Bangguo Li, Hengkai Tan, Huayu Chen, Zhengyi Wang, Ke Xu, Hang Su, and Jun Zhu. RDT-1B: a diffusion foundation model for bimanual manipulation. In *International Conference on Learning Representations*, 2025.
- [6] Songming Liu, Bangguo Li, Kai Ma, Lingxuan Wu, Hengkai Tan, Xiao Ouyang, Hang Su, and Jun Zhu. RDT2: Exploring the scaling limit of umi data towards zero-shot cross-embodiment generalization. In *Forty-third International Conference on Machine Learning*, 2026.
- [7] Motubrain Team, Chendong Xiang, Fan Bao, Haitian Liu, Hengkai Tan, Hongzhe Bi, James Li, Jiabao Liu, Jingrui Pang, Kiro Jing, Louis Liu, Mengchen Cai, Rongxu Cui, Ruowen Zhao, Runqing Wang, Shuhe Huang, Yao Feng, Yinze Rong, Zeyuan Wang, and Jun Zhu. MotuBrain: An advanced world action model for robot control. *arXiv preprint arXiv:2604.27792*, 2026.
- [8] Yao Feng, Hengkai Tan, Xinyi Mao, Chendong Xiang, Guodong Liu, Shuhe Huang, Hang Su, and Jun Zhu. ViDAR: Embodied video diffusion model for generalist manipulation, 2025.
- [9] Jun Zhu, Hengkai Tan, Jintao Zhang, Min Zhao, Fan Bao, and Bo Zhang. General world models from first-principles, 2026. Manuscript.
- [10] Fan Bao, Chendong Xiang, Gang Yue, Guande He, Hongzhou Zhu, Kaiwen Zheng, Min Zhao, Shilong Liu, Yaole Wang, and Jun Zhu. Vidu: a highly consistent, dynamic and skilled text-to-video generator with diffusion models. *arXiv preprint arXiv:2405.04233*, 2024.
- [11] Fan Bao, Shen Nie, Kaiwen Xue, Yue Cao, Chongxuan Li, Hang Su, and Jun Zhu. All are worth words: A vit backbone for diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 22669–22679, 2023.
- [12] Fan Bao, Chongxuan Li, Jun Zhu, and Bo Zhang. Analytic-DPM: An analytic estimate of the optimal reverse variance in diffusion probabilistic models. In *International Conference on Learning Representations*, 2022.
- [13] Cheng Lu, Yuhao Zhou, Fan Bao, Jianfei Chen, Chongxuan Li, and Jun Zhu. Dpm-solver: A fast ode solver for diffusion probabilistic model sampling in around 10 steps. *Advances in neural information processing systems*, 35:5775–5787, 2022.
- [14] Huayu Chen, Cheng Lu, Chengyang Ying, Hang Su, and Jun Zhu. Offline reinforcement learning via high-fidelity generative behavior modeling. In *International Conference on Learning Representations*, 2023.
- [15] Cheng Lu, Huayu Chen, Jianfei Chen, Hang Su, Chongxuan Li, and Jun Zhu. Contrastive energy prediction for exact energy-guided diffusion sampling in offline reinforcement learning. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 22825–22855, 2023.
- [16] Huayu Chen, Cheng Lu, Zhengyi Wang, Hang Su, and Jun Zhu. Score regularized policy optimization through diffusion behavior. In *International Conference on Learning Representations*, 2024.

-
- [17] Huayu Chen, Kaiwen Zheng, Hang Su, and Jun Zhu. Aligning diffusion behaviors with q-functions for efficient continuous control. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
 - [18] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, You Liang Tan, Chuning Zhu, Jiannan Xiang, Ayaan Malik, Kyungmin Lee, William Liang, Nadun Ranawaka, Jiasheng Gu, Yinzhen Xu, Guanzhi Wang, Fengyuan Hu, Avnish Narayan, Johan Bjorck, Jing Wang, Gwanghyun Kim, Dantong Niu, Ruijie Zheng, Yuqi Xie, Jimmy Wu, Qi Wang, Ryan Julian, Danfei Xu, Yilun Du, Yevgen Chebotar, Scott Reed, Jan Kautz, Yuke Zhu, Linxi "Jim" Fan, and Joel Jang. World action models are zero-shot policies, 2026.
 - [19] Shenyuan Gao, William Liang, Kaiyuan Zheng, Ayaan Malik, Seonghyeon Ye, Sihyun Yu, Wei-Cheng Tseng, Yuzhu Dong, Kaichun Mo, Chen-Hsuan Lin, Qianli Ma, Seungjun Nah, Loic Magne, Jiannan Xiang, Yuqi Xie, Ruijie Zheng, Dantong Niu, You Liang Tan, K. R. Zentner, George Kurian, Suneel Indupuru, Pooya Jannaty, Jinwei Gu, Jun Zhang, Jitendra Malik, Pieter Abbeel, Ming-Yu Liu, Yuke Zhu, Joel Jang, and Linxi Fan. DreamDojo: A generalist robot world model from large-scale human videos. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306, 2026. Spotlight.
 - [20] Ruihan Yang, Qinxu Yu, Yecheng Wu, Rui Yan, Borui Li, An-Chieh Cheng, Xueyan Zou, Yunhao Fang, Xuxin Cheng, Ri-Zhao Qiu, Hongxu Yin, Sifei Liu, Song Han, Yao Lu, and Xiaolong Wang. EgoVLA: Learning vision-language-action models from egocentric human videos. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2026.
 - [21] Ruijie Zheng, Dantong Niu, Yuqi Xie, Jing Wang, Mengda Xu, Yunfan Jiang, Fernando Castaneda, Fengyuan Hu, You Liang Tan, Letian Fu, Trevor Darrell, Furong Huang, Yuke Zhu, Danfei Xu, and Linxi Fan. EgoScale: Scaling dexterous manipulation with diverse egocentric human data, 2026.
 - [22] Dantong Niu, Zhuoyang Liu, Zekai Wang, Boning Shao, Zhao-Heng Yin, Anirudh Pai, Yuvan Sharma, Stefano Saravalle, Ruijie Zheng, Jing Wang, et al. T-rex: Tactile-reactive dexterous manipulation. *arXiv preprint arXiv:2606.17055*, 2026.
 - [23] Zhenyu Jiang, Yuqi Xie, Kevin Lin, Zhenjia Xu, Weikang Wan, Ajay Mandlekar, Linxi Fan, and Yuke Zhu. Dexmimicgen: Automated data generation for bimanual dexterous manipulation via imitation learning. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025.
 - [24] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An open-source vision-language-action model. In *Proceedings of the 8th Conference on Robot Learning*, volume 270, pages 2679–2713, 2025.
 - [25] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lindsay Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Huamin Wang, and Ury Zhilinsky. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
 - [26] Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Manuel Y Galliker, et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. In *Proceedings of the 9th Conference on Robot Learning*, volume 305, pages 17–40, 2025.
 - [27] Zongzheng Zhang, Jingrui Pang, Zhuo Yang, Kun Li, Minwen Liao, Saining Zhang, Guoxuan Chi, Jinbang Guo, Huanang Gao, Modi Shi, Dongyun Ge, Yao Mu, Jiayuan Gu, Rui Chen, Hao Dong, Huazhe Xu, Li Yi, Yixin Zhu, Hang Zhao, Pengwei Wang, Shanghang Zhang, Guocai Yao, Jianyu Chen, Hongyang Li, and Hao Zhao. Dexora: Open-Source VLA for High-DoF Bimanual Dexterity. In *2026 IEEE International Conference on Robotics and Automation (ICRA)*, 2026.
 - [28] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems*, 2023.
 - [29] Kristen Grauman, Andrew Westbury, Eugene Byrne, Zachary Chavis, Antonino Furnari, Rohit Girdhar, Jackson Hamburger, Hao Jiang, Miao Liu, Xingyu Liu, Miguel Martin, Tushar Nagarajan, Ilija Radosavovic, Santhosh K. Ramakrishnan, Fiona Ryan, Jayant Sharma, Michael Wray, Mengmeng Xu, Eric Zhongcong Xu, Chen Zhao, Siddhant Bansal, et al. Ego4D: Around the world in 3,000 hours of egocentric video. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 18995–19012, 2022.

-
- [30] Sizhe Yang, Juncheng Mu, Tianming Wei, Chenhao Lu, Xiaofan Li, Linning Xu, Zhengrong Xue, Zhecheng Yuan, Dahua Lin, Jiangmiao Pang, and Huazhe Xu. MemoryWAM: Efficient world action modeling with persistent memory. *arXiv preprint arXiv:2606.20562*, 2026.
 - [31] Yicong Hong, Yiqun Mei, Chongjian Ge, Yiran Xu, Yang Zhou, Sai Bi, Yannick Hold-Geoffroy, Mike Roberts, Matthew Fisher, Eli Shechtman, et al. Relic: Interactive video world model with long-horizon memory. *arXiv preprint arXiv:2512.04040*, 2025.
 - [32] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
 - [33] Tim Brooks, Bill Peebles, Connor Holmes, Will DePue, Yufei Guo, Li Jing, David Schnurr, Joe Taylor, Troy Luhman, Eric Luhman, Clarence Ng, Ricky Wang, and Aditya Ramesh. Video generation models as world simulators. 2024.
 - [34] Jake Bruce, Michael D Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, et al. Genie: Generative interactive environments. In *Forty-first International Conference on Machine Learning*, 2024.
 - [35] Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-WAM: Do world action models need test-time future imagination?, 2026.
 - [36] Hao Luo, Wanpeng Zhang, Yicheng Feng, Sipeng Zheng, Haiweng Xu, Chaoyi Xu, Ziheng Xi, Yuhui Fu, and Zongqing Lu. Being-H0.7: A latent world-action model from egocentric videos. *arXiv preprint arXiv:2605.00078*, 2026.
 - [37] Dyna Robotics. Dyna-2: A 1-million-hour scaling law for world-action models. Technical report, August 2026.
 - [38] Yanjiang Guo, Lucy Xiaoyang Shi, Jianyu Chen, and Chelsea Finn. Ctrl-World: A controllable generative world model for robot manipulation. In *International Conference on Learning Representations*, 2026.
 - [39] GigaWorld Team, Angyuan Ma, Boyuan Wang, Bohan Li, Chaojun Ni, Guo Li, Guan Huang, Guosheng Zhao, Hao Li, Hengtao Li, Jingyu Liu, Jiwen Lu, Qiuping Deng, Tingdong Yu, Xuancheng Xu, Xinyu Zhou, Xiuwei Xu, Xinze Chen, Xiaofeng Wang, Xiaoyu Tian, Yang Wang, Yifan Chang, Yukun Zhou, Yun Ye, Zhenyu Wu, Zhanqian Wu, and Zheng Zhu. GigaWorld-1: A roadmap to build world models for robot policy evaluation. *arXiv preprint arXiv:2607.02642*, 2026.
 - [40] Zhihao Wang, Jianxiong Li, Yu Cui, Yuan Gao, Xianyu Zhan, Junzhi Yu, and Xiao Ma. World value models for robotic manipulation, 2026.
 - [41] Yankai Fu, Ning Chen, Junkai Zhao, Shaozhe Shan, Guocai Yao, Pengwei Wang, Zhongyuan Wang, and Shanghang Zhang. Metis: Multi-source egocentric training for integrated dexterous vision-language-action model. *arXiv preprint arXiv:2511.17366*, 2025.
 - [42] Yifan Zhong, Zhang Chen, Tianrui Guan, Fanlian Zeng, Yuyao Ye, Tianjia He, Ka Nam Lui, Jiayi Li, Tingrui Zhang, Ruilin Yan, Xinhao Ji, Guangyu Zhao, Wenjie Lou, Jiayuan Zhang, Yuanpei Chen, and Yaodong Yang. Egosteer: A full-stack system towards steerable dexterous manipulation from egocentric videos. *arXiv preprint arXiv:2607.09701*, 2026.
 - [43] Baoyu Li, Xinchun Yin, Mengying Lin, Yixin Zhang, and Danfei Xu. Egowam: World action models beyond pixels with in-the-wild egocentric human data. *arXiv preprint arXiv:2607.08436*, 2026.
 - [44] Fangqi Zhu, Zhengyang Yan, Zicong Hong, Quanxin Shou, Xiao Ma, and Song Guo. WMPO: World model-based policy optimization for vision-language-action models. In *International Conference on Learning Representations*, 2026.
 - [45] Jiazhi Yang, Kunyang Lin, Jinwei Li, Wencong Zhang, Tianwei Lin, Longyan Wu, Zhizhong Su, Hao Zhao, Ya-Qin Zhang, Li Chen, Ping Luo, Xiangyu Yue, and Hongyang Li. RISE: Self-improving robot policy with compositional world model. In *Robotics: Science and Systems*, 2026.
 - [46] Jiahui Zhang, Ze Huang, Chun Gu, Zipei Ma, and Li Zhang. Reinforcing action policies by prophesying. *arXiv preprint arXiv:2511.20633*, 2025.
 - [47] Chia-Yu Hung, Navonil Majumder, Haoyuan Deng, Liu Renhang, Yankang Ang, Amir Zadeh, Chuan Li, Dorien Herremans, Ziwei Wang, and Soujanya Poria. Nora-1.5: A vision-language-action model trained using world model- and action-based preference rewards. *arXiv preprint arXiv:2511.14659*, 2025.

-
- [48] Team Wan, Ang Wang, Baole Ai, Bin Wen, Chaojie Mao, Chen-Wei Xie, Di Chen, Feiwu Yu, Haiming Zhao, Jianxiao Yang, et al. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025.
- [49] Angen Ye, Boyuan Wang, Chaojun Ni, Guan Huang, Guosheng Zhao, Hao Li, Hengtao Li, Jie Li, Jindi Lv, Jingyu Liu, Min Cao, Peng Li, Qiuping Deng, Wenjun Mei, Xiaofeng Wang, Xinze Chen, Xinyu Zhou, Yang Wang, Yifan Chang, Yifan Li, Yukun Zhou, Yun Ye, Zhichao Liu, and Zheng Zhu. GigaWorld-Policy: An efficient action-centered world-action model. *arXiv preprint arXiv:2603.17240*, 2026.
- [50] Qi Zhang, Shaopeng Zhai, Shengzhe Zhang, Litao Liu, Tianyi Zhang, Fuxian Huang, and Ming Zhou. A generalist pair-wise progress critic model for vision-language-action robots. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306, 2026.
- [51] Chengxuan Lu, Shukuan Wang, Yanjie Li, Yingying Fang, Huoyan Wang, Tian Zhang, Wei Liu, Shiji Jin, Fuyuan Qian, Peiming Li, Chao Xu, Baigui Sun, and Yang Liu. Accerl: A distributed asynchronous reinforcement learning and world model framework for vision-language-action models. *arXiv preprint arXiv:2603.18464*, 2026.

A. Robot System and Teleoperation Stack

A.1. Robot Platforms

We use three bimanual robot configurations for data collection and post-training. The first combines a Tianji Marvin dual-arm system with two 20-DoF Wuji dexterous hands. This platform, shown in Fig. 6(a), additionally integrates ZED Mini and ZED 2 cameras and Wuji Robot Glove tactile sensing. The other two configurations both use the Tianji Gento Luna dual-arm system: one is equipped with 20-DoF Wuji Hand 2, and the other with 22-DoF Sharpa Wave; these two configurations are summarized in Fig. 6.



(a) Wuji hands with Tianji Marvin dual arms.



(b) Wuji Hand 2 with Tianji Gento Luna.



(c) Sharpa Wave hands with Tianji Gento Luna.



(d) Human-robot alignment setup.

Figure 6: Robot platforms and human-robot alignment collection. (a) Wuji hands with Tianji Marvin dual arms. (b) Wuji Hand 2 with Tianji Gento Luna. (c) Sharpa Wave hands with Tianji Gento Luna. (d) Human-robot alignment collection setup.

A.2. Teleoperation

The three systems use platform-specific control interfaces while retaining a common separation between teleoperation, recording control, and data storage. On the Marvin-Wuji platform, MANUS glove observations are first converted to a 21-keypoint MediaPipe hand representation and then retargeted to the Wuji joint space. Three VIVE Ultimate Trackers provide torso and bilateral wrist poses. Relative wrist motion is converted into dual-arm end-effector targets, which are solved by the Tianji arm inverse-kinematics controller. On both Gento Luna platforms, the left and right PICO controller poses define the dual-arm targets. Glove observations are independently retargeted to the corresponding Wuji or Sharpa hand.

B. Human-Robot Alignment Data Collection

Human-robot alignment data are collected without commanding a robot. The operator performs manipulation tasks while wearing a pair of Wuji Human Gloves and one HTC VIVE Tracker on each wrist. A third tracker is

fixed to the table as a stationary workspace reference. A ZED 2 stereo camera records the first-person left and right RGB views.

The table-fixed tracker defines the reference frame used for alignment. Let ${}^V\mathbf{T}_F$ and ${}^V\mathbf{T}_{W_h}$ denote the fixed tracker and wrist poses in the VIVE Lighthouse frame for hand $h \in \{L, R\}$. The wrist pose expressed in the fixed reference frame is

$${}^F\mathbf{T}_{W_h} = ({}^V\mathbf{T}_F)^{-1} {}^V\mathbf{T}_{W_h}. \quad (19)$$

This representation removes the arbitrary Lighthouse origin and expresses human wrist motion in a stable workspace-centered frame.

C. Egocentric Data Curation and Offline Annotation

We convert recordings with hand-pose observations into a unified representation through pose standardization, quality-aware temporal segmentation, and language annotation. Each processed episode contains synchronized RGB streams, hand poses, and natural-language annotations. When multiple views or auxiliary modalities are available, identical temporal boundaries are applied to every signal to preserve frame-level synchronization.

C.1. Pose Standardization

Hand observations are represented by a 134-dimensional vector:

$$\mathbf{e}_t = [\mathbf{p}_t^L, \mathbf{q}_t^L, \mathbf{k}_t^L, \mathbf{p}_t^R, \mathbf{q}_t^R, \mathbf{k}_t^R], \quad (20)$$

where $\mathbf{p}_t^h \in \mathbb{R}^3$ and $\mathbf{q}_t^h \in \mathbb{R}^4$ denote the world-frame wrist translation and an $xyzw$ quaternion for hand h , respectively. $\mathbf{k}_t^h \in \mathbb{R}^{60}$ contains the wrist-local coordinates of 20 non-wrist hand keypoints. The wrist keypoint is not repeated in $\mathbf{k}_t^h$.

When wrist poses are not already expressed in a common world coordinate system, available camera trajectories and calibration parameters transform them into the world frame. Finger coordinates are then expressed relative to the corresponding wrist. Original RGB streams are retained, while available depth, inertial, calibration, camera-pose, and motion-capture signals are preserved as synchronized auxiliary information.

C.2. Quality-Aware Temporal Processing

To curate long and noisy ego-centric recordings, we design a quality-aware temporal processing pipeline, termed BoxTrim. BoxTrim removes unreliable temporal regions and converts the remaining recordings into compact, action-centric clips while preserving synchronization across all available modalities.

The pipeline targets three common failure modes in ego-centric recordings: (i) rapid body turns that induce abrupt viewpoint changes, (ii) large translational movements or discontinuities in reconstructed hand trajectories, and (iii) intervals in which both hands leave the observable field. Because reliable camera-pose signals are not consistently available for every source, BoxTrim uses wrist-trajectory discontinuities as a unified proxy for rapid turns and large ego-centric motion. Simultaneous invalidity of the two hand-keypoint tracks is used as a proxy for both hands leaving the observable field or for complete hand-tracking failure.

Let $\mathbf{p}_t^L$ and $\mathbf{p}_t^R$ denote the reconstructed positions of the left and right wrists. We define the instantaneous motion magnitude as

$$d_t = \max(\|\mathbf{p}_{t+1}^L - \mathbf{p}_t^L\|_2, \|\mathbf{p}_{t+1}^R - \mathbf{p}_t^R\|_2). \quad (21)$$

An adaptive threshold is estimated from the empirical wrist-motion distribution of each recording:

$$\tau_{\text{motion}} = \max(\alpha \text{ median}\{d_t \mid d_t > 0\}, \tau_{\text{min}}), \quad (22)$$

where α is a robust scale factor and τ_{min} prevents nearly static recordings from producing an overly sensitive threshold. Frames satisfying $d_t > \tau_{\text{motion}}$ are treated as motion discontinuities. A short temporal margin is

added on both sides of every detected event to remove neighboring frames that may also contain corrupted pose estimates.

To identify intervals in which the hands are not reliably observed, let $\mathbf{k}_t^h$ denote the finger-keypoint vector of hand h . We define

$$z_t^h = \mathbb{I}[\|\mathbf{k}_t^h\|_1 < \epsilon \vee \neg \text{Finite}(\mathbf{k}_t^h)], \quad (23)$$

and mark a frame as unreliable when both hand tracks are invalid:

$$b_t = z_t^L \wedge z_t^R. \quad (24)$$

Sustained runs of $b_t = 1$ are interpreted as intervals in which both hands have left the observable field or the hand tracker has otherwise failed. This pose-space criterion does not require an explicit image-plane hand detector.

The high-motion and invalid-hand intervals are merged and removed from the original recording. The same temporal mask is applied to all synchronized RGB streams, pose trajectories, depth observations, calibration data, and auxiliary motion signals. When semantic temporal annotations are available, their timestamps are first mapped to video-frame indices and used as preferred clip boundaries. Overlapping intervals are merged, and long intervals are further divided near low-motion frames. Very short fragments are discarded, while long clips are subdivided into more balanced temporal chunks. Residual non-finite values are sanitized during a final consistency pass, and all resulting clips are deterministically renumbered.

The exact temporal thresholds are selected according to the frame rate, annotation density, and tracking characteristics of each data source. We do not assume a universal frame rate or apply a single set of frame-count thresholds to all corpora.

C.3. Language Annotation

After conversion, each clip is annotated with a vision-language model. For recordings with multiple synchronized views, one consistently selected egocentric RGB stream is used for caption generation while all views remain in the processed data. The model returns structured descriptions of (i) the environment, (ii) the overall manipulation behavior, and (iii) left-right differentiated hand behavior.

D. Hand Retargeting Algorithm

The egocentric human data used for pretraining represent each hand using 21 MediaPipe-format 3D keypoints. We convert these keypoints offline to the 20-DoF joint space of the first-generation Wuji hand. Let $\mathbf{X}_t^h \in \mathbb{R}^{21 \times 3}$ denote the keypoints of hand $h \in \{L, R\}$ at time t , with keypoint 0 corresponding to the wrist. We first express all keypoints relative to the wrist. A wrist-centered hand frame is then estimated from the wrist, index-finger MCP, and middle-finger MCP landmarks. Specifically, the palm normal is estimated by singular value decomposition, and the remaining axes are constructed from the wrist-to-middle-MCP direction and the palm normal. A side-specific rigid rotation maps the left- and right-hand observations into the corresponding Wuji model coordinates.

For each valid frame, we solve an inverse-kinematics problem using the Wuji URDF model. Let $\mathbf{q} \in \mathbb{R}^{20}$ denote a candidate Wuji joint-angle vector, $\mathbf{q}_t^*$ the optimized joint-angle vector at time t , and $\mathbf{q}_{\min}, \mathbf{q}_{\max} \in \mathbb{R}^{20}$ the joint-limit vectors. We define the feasible joint space as $\mathcal{Q} = \{\mathbf{q} \mid \mathbf{q}_{\min} \leq \mathbf{q} \leq \mathbf{q}_{\max}\}$, where the inequalities are applied elementwise. Let $\mathcal{F}$ denote the set of five fingers. For compactness, we write $\mathcal{L}_{\text{tip},t}^f(\mathbf{q}) = \mathcal{L}_{\text{tip}}^f(\mathbf{q}, \mathbf{X}_t^h)$ and $\mathcal{L}_{\text{full},t}^f(\mathbf{q}) = \mathcal{L}_{\text{full}}^f(\mathbf{q}, \mathbf{X}_t^h)$. The implemented objective is

$$\mathbf{q}_t^* = \arg \min_{\mathbf{q} \in \mathcal{Q}} \sum_{f \in \mathcal{F}} \left[\alpha_{t,f} \mathcal{L}_{\text{tip},t}^f(\mathbf{q}) + (1 - \alpha_{t,f}) \mathcal{L}_{\text{full},t}^f(\mathbf{q}) \right] + \lambda \|\mathbf{q} - \mathbf{q}_{t-1}^*\|_2^2. \quad (25)$$

The objective combines two losses with different geometric roles. The full-finger loss $\mathcal{L}_{\text{full},t}^f$ preserves the overall configuration of finger f . It compares three vectors measured from the human wrist with their

robot counterparts measured from the robot palm: the vectors to the proximal joint, the distal joint, and the fingertip. For the four non-thumb fingers, these points correspond to the PIP, DIP, and fingertip landmarks; for the thumb, the MCP, IP, and fingertip landmarks are used. The human vectors are rescaled to account for differences between human and robot finger lengths, while the robot vectors are computed through forward kinematics. The loss is the average Huber penalty over the three vector discrepancies.

The fingertip loss $\mathcal{L}_{\text{tip},t}^f$ places greater emphasis on the fingertip geometry. It compares both the fingertip position relative to the wrist or robot palm and the direction of the final finger segment. The human segment direction is measured from the DIP landmark to the fingertip, or from the IP landmark to the fingertip for the thumb. The corresponding robot direction is obtained from the final finger link through forward kinematics. Huber penalties on the fingertip-position and direction discrepancies are combined to form $\mathcal{L}_{\text{tip},t}^f$.

The interpolation weight $\alpha_{t,f}$ determines the relative importance of the two losses. For each non-thumb finger, it is computed from the distance between that fingertip and the thumb tip. As the two fingertips approach, $\alpha_{t,f}$ increases and places more emphasis on the fingertip loss. Otherwise, the full-finger loss receives more weight and preserves the overall finger configuration. The thumb weight is set to the maximum weight among the other four fingers. The weights are clipped to a bounded range so that both losses remain active. The coefficient λ controls temporal regularization toward the previous solution $\mathbf{q}_{t-1}^*$.

Non-finite or geometrically degenerate keypoint frames are removed before optimization. The optimization is constrained by the Wuji joint limits, and a first-order low-pass filter is applied to the optimized trajectory. The result contains 20 finger joint angles per hand and forms the hand-joint component of the pretraining action representation.

This retargeting procedure is applied only to human keypoint data used for pretraining. For post-training data collected with the 20-DoF Wuji Hand 2 and the 22-DoF Sharpa Wave hands, we directly use the joint angles recorded by the corresponding robot data-collection stacks in their native joint spaces. These joint angles are not produced by the MediaPipe-to-Wuji retargeting procedure described above.

E. Model and Training Details

Notation. A *latent frame* is the spatial video representation produced by the video encoder at one latent timestep. We use W for the total number of latent frames in one sampled training window and κ for the number of consecutive latent frames grouped into one prediction chunk; the aligned action segment is grouped into the same chunk. A history width measured in chunks therefore corresponds to κ times as many latent frames.

E.1. Pre-training

Stage 1: video-only pre-training. We initialize the visual backbone from Wan 2.2-TI2V-5B and train only the video pathway with conditional flow matching. Stage 1 first uses low-resolution monocular egocentric clips for 500K steps and then high-resolution monocular clips for 340K steps. Each sample provides one or two clean latent frames as context. Action and value tokens are absent, and the later action-first mask is not used.

Stage 2: joint video-action pre-training. Stage 2 starts from the video-only checkpoint and trains for 450K steps on stereo egocentric video paired with human actions. Stereo views share temporal and vertical RoPE coordinates and use offset horizontal ranges. Video and action tokens are mutually visible within a chunk and are jointly supervised by flow matching; the value loss remains disabled. The maximum training window contains $W = 12$ latent frames, each prediction chunk contains $\kappa = 2$ frames, and the visible clean-history width is sampled uniformly from one to five chunks. At streaming inference the history width is fixed to two chunks. Stage 2 uses AdamW with learning rate 2×10^{-5} , weight decay 0.01, a constant schedule with 1K warm-up steps, gradient clipping at 0.5, and bfloat16 training.

E.2. Mid-training

Mid-training adapts the Stage 2 checkpoint to the robot domain using more than 100 hours of data: robot trajectories supplemented with human-robot alignment data. We switch from the joint within-chunk mask to the action-first mask and mix the policy, simulation, and evaluation modes in §3.2. The robot-domain recipe uses 640×384 stereo inputs, $W = 8$ latent frames, $\kappa = 2$ frames per chunk, a training history of one to three chunks, and a two-chunk streaming cache. Simulation- and evaluation-mode samples account for 0.10 and 0.05 of training samples, respectively; the remaining samples use policy mode. The value target is discretized into 201 bins with loss weight 0.05. We use AdamW with learning rate 2×10^{-5} , weight decay 0.01, 200 warm-up steps, a linear schedule, gradient clipping at 0.5, and bfloat16 training.

E.3. Post-training

Post-training starts from the mid-trained checkpoint and specializes the model to target-robot control. Supervised adaptation retains the action-first mask; the branches below change the temporal context or add the self-evolution and tactile objectives.

Sliding-window SFT. The default target-robot policy uses $W = 8$, $\kappa = 2$, a training history sampled from one to three chunks, and a two-chunk streaming cache. Temporal RoPE is window-relative, so surviving video keys are rebased after cache eviction. Action chunks are sampled with five flow-matching denoising steps during default deployment.

Global autoregression. The full-history branch trains on complete episodes with global temporal RoPE and a video-shared latent clock for action tokens. It disables compressed memory by setting the numbers of anchor and memory tokens to zero and makes the recent window non-binding, so every prediction chunk can attend to all preceding clean observations. This branch therefore retains NL visual KV tokens after N latent frames.

Hybrid working memory. Our MemoryWAM adaptation uses two full-resolution anchor frames, four full-resolution recent frames, and eight memory tokens per latent frame. It is trained on complete episodes with global temporal RoPE and video-shared action positions. In the reported long-context comparison, the global-autoregressive and hybrid branches use 10K optimization steps, learning rate 2×10^{-5} , AdamW betas (0.9, 0.95), weight decay 0.01, 200 warm-up steps, gradient clipping at 0.5, and 50 action-denoising steps for evaluation. Their variable-length loader uses a latent-frame budget of 100 and packs at most eight complete episodes per micro-batch; an episode exceeding the budget is kept intact as a singleton rather than truncated.

MBRL and planning. For each sampled prefix, the rollout pipeline generates eight policy candidates and adds one ground-truth candidate. Prefixes contain one to four chunks and the imagined rollout horizon is one chunk. In the asynchronous joint-rollout pipeline, action and future-video sampling use four and eight denoising steps, respectively. DiffusionNFT uses $\beta = 0.1$ and one sampled noise level per candidate. The EMA reference has decay 0.99 and is published every ten trainer updates. Only action-related parameters are optimized; the video backbone and value components remain frozen. Rollouts from stale reference versions or more than 600 seconds old are discarded.

Tactile expert. We use a common tactile-expert architecture and temporal schedule for the Sharpa Wave and Wuji Hand 2 dexterous hands. The expert is a 30-layer transformer with hidden width 128, four attention heads, and feed-forward width 512. Its layers are paired one-to-one with the shared backbone: each layer self-attends over tactile and intermediate-action tokens and reads the cached K/V states from the corresponding backbone layer. The backbone denoises each 48-action chunk to $\sigma_c = 0.2$ once, then refreshes the action K/V at this intermediate state and concatenates them with the visible video-context K/V. The resulting layer-wise cache is detached and reused by the tactile expert. At the action rate of 30 Hz, the chunk is divided into

eight six-action sub-chunks, each spanning 0.2 seconds. The expert refines these sub-chunks sequentially using the immediately preceding tactile window. Force signals are sampled at 90 Hz, while the Sharpa Wave additionally provides six deformation frames per window. The same intermediate action chunk and KV cache are reused across all eight calls. The intermediate actions and cached states are detached so that the tactile losses update only the expert, the force encoder and decoder, and, for the Sharpa Wave, the deformation encoder.

Sharpa Wave. The expert receives a 60-dimensional bimanual force–torque vector together with deformation images from ten finger pads. A temporal convolutional encoder maps six consecutive deformation frames to 20 conditioning tokens. The deformation images are not prediction targets.

Wuji Hand 2. The expert receives a 1086-dimensional bimanual vector containing 1056 pointwise force values from 176 three-axis sensing points per hand and 30 per-finger aggregate three-axis force values.

For both dexterous hands, force inputs are normalized using the configured training-data statistics and invalid channels are set to zero. For force prediction, the immediately preceding force window is encoded without noise and used as a conditioning state; the Sharpa Wave additionally conditions on the preceding deformation images. The real following force window is corrupted in normalized signal space and passed through the same force encoder. The expert performs conditional flow matching over the resulting tokens, and a force decoder maps the future-force token states back to a velocity in force space. The prediction loss is evaluated only against the flow target constructed from the real following force signal. We jointly optimize the expert, force encoder and decoder, and the Sharpa deformation encoder using Eqs. (15) and (16), with $\lambda_{\text{pred}} = 0.1$. Optimization uses AdamW with a learning rate of 5×10^{-5} and weight decay 0.01.

F. Qualitative Value-Model Trajectories

Figures 7a and 7b show predicted task progress along recorded trajectories from four tasks that differ in horizon, in the manipulated object, and in their failure modes. Each task contributes one successful and one failed execution, and all eight panels are scored by the same value model.

Protocol. Values are read out in the evaluation mode of Eq. (7), so the prediction for chunk t is conditioned on the context, the executed actions and the observed video of that chunk, and never on the outcome: whether the episode eventually succeeded is not provided to the model. Each recorded episode is replayed through the streaming window used during training, and one value is read per latent frame rather than per video frame, which sets the spacing of the plotted points. Values follow the relative-progress reward of Eq. (8) and Eq. (9) and therefore lie in $[-1, 1]$; they are trained to rank rollouts relative to one another and are not calibrated success probabilities. Keyframes above each panel are timestamped and show the left-camera view only, and all eight panels share the same vertical scale.

Successful executions. Panels (a), (c), (e) and (g) terminate between 0.63 and 0.65 but follow distinct profiles. On Cut Reeds the prediction remains near zero for the first 15 s, during which the hand approaches and aligns but no reed has been cut, and then increases in two steps as the cuts are completed. On Make Tea, Multi-Finger and Put Phone the prediction increases smoothly from the outset, consistent with tasks in which most of the motion contributes incrementally to completion. In all four panels the predicted level alone indicates how far the task has progressed.

Failed executions. The failed executions are not uniformly low, and their shape is the more informative signal. On Cut Reeds (b) the prediction follows its successful counterpart into the cutting phase and reaches 0.27. The first cut then fails to sever the reed and lifts it instead, and the prediction falls to -0.39 within two seconds; it declines further to -0.51 as the rack holding the reeds is knocked out of alignment. Put Phone (h) exhibits the largest excursion: it peaks at 0.60 once the phone has been placed inside the box, and decreases to -0.26 only after the robot has repeatedly failed to close the lid, which the task requires. On Make Tea (d) the prediction rises to 0.14 at approximately 35 s, by which point the water has been poured into the cup

holding the tea leaves, and then decreases over the remaining 80 s to -0.47 as the robot fails to lift the lid off the table. Multi-Finger (f) rises to 0.18 near 50 s and falls to -0.53 at the end of the episode: the task requires the objects held in the multi-finger grasp to be placed in separate containers, and here they are all released into the same one. This common structure—an increase while the early portion of the task is executed correctly, followed by a decrease once progress stops—is what allows the value model to distinguish a recoverable state from an unrecoverable one when MBRL scores imagined rollouts.

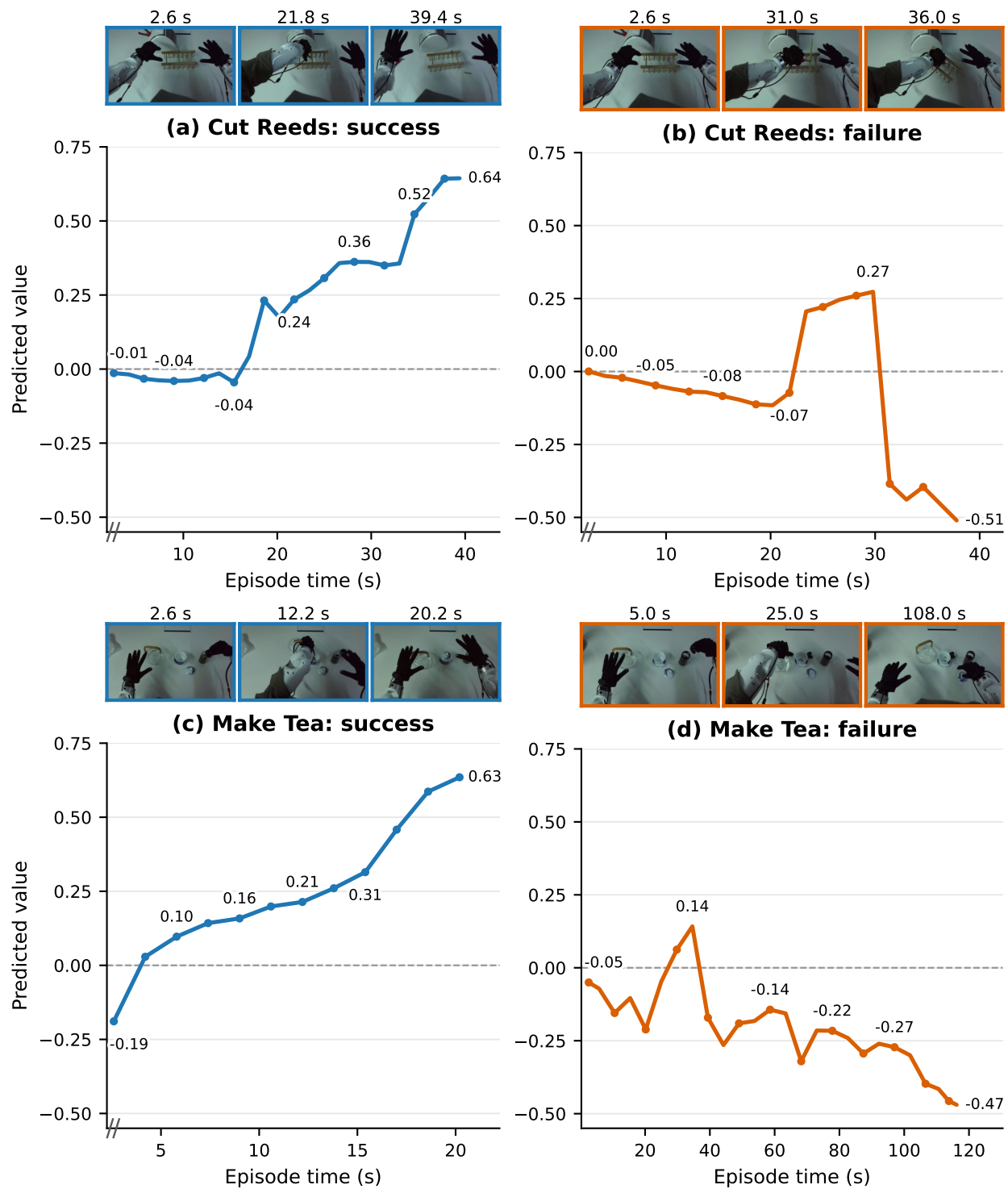


Figure 7a: Predicted task progress: Cut Reeds and Make Tea.

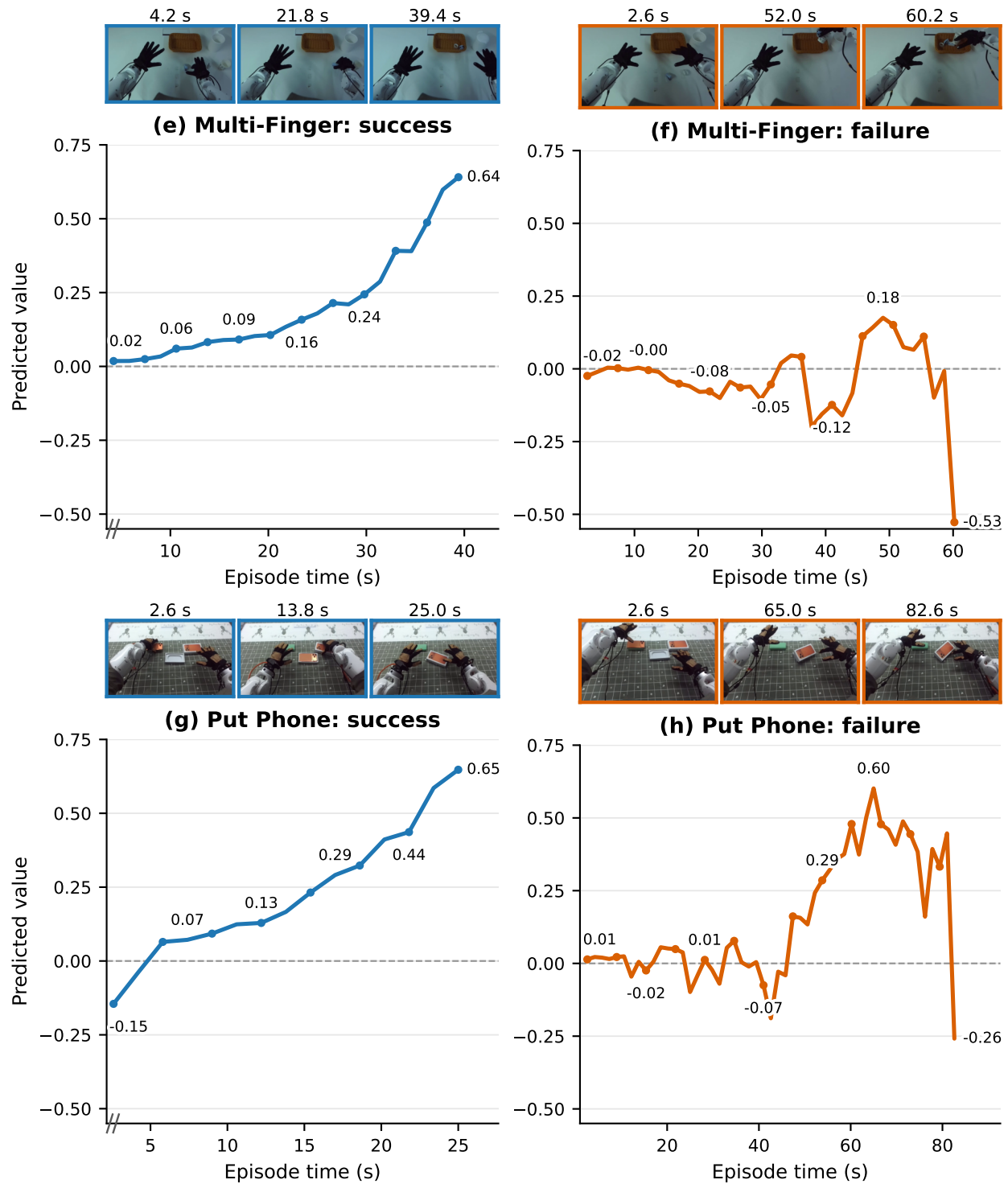


Figure 7b: Predicted task progress: Multi-Finger and Put Phone.

G. Evaluation Tasks

Figure 8 summarizes the nine unique physical tasks. The MBRL study reuses two tasks from the main suite rather than introducing additional task definitions.



(a) Place Ball. The robot uses its left hand to pick up a ball and place it on top of an inverted cup.



(b) Put Phone. The robot uses its right hand to pick up a smartphone from a purple block and place it into a white phone stand.



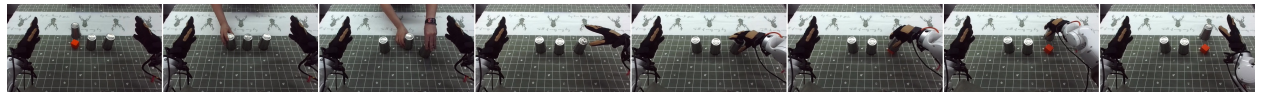
(c) Attach Eraser. The robot uses its left hand to pick up an eraser and attach it to the whiteboard surface.



(d) Screw Bulb. The robot uses its right hand to screw a bulb into a socket until it lights, then releases it.



(e) Multi-Finger. The robot grasps an object from a set of differently shaped items and places it into an orange basket.



(f) Find Square. After a person hides an orange square under one of three cups and shuffles them, the robot must use the earlier observation to lift the correct cup.



(g) Press Button. The robot observes a visual cue, retains it while acting, and presses the indicated physical button sequence.



(h) Pull Out Paper Cup. The robot stabilizes a nested stack and extracts a single paper cup without dropping or crushing it.



(i) Tear Paper. The robot coordinates both hands to grasp and tear a sheet of paper from the roll.

Figure 8: Evaluation tasks. Each row shows eight frames sampled from a representative episode. **(a)–(e)** are the main-suite tasks, **(f)–(g)** are the working-memory tasks, and **(h)–(i)** are the tactile tasks. The MBRL study reuses Put Phone and Multi-Finger from the main suite under different policy-optimization and inference settings.